

Architettura degli Elaboratori

Elena Giunta – A.A. 2023-2024 Canale F-N

1.1 Famiglie di calcolatori

Esistono 4 tipi di calcolatori:

- *Embedded/incorporati* : programmati per uno scopo specifico. Difficilmente possono essere riprogrammati.
- *Calcolatori personali*: Desktop e laptop.
- *Server*: In grado di immagazzinare enormi quantità di richieste, comunica dati offrendo servizi ad altri computer
- *Supercalcolatori*: Grid Computing o Cloud Computing, prestazioni elevate, per simulazioni complesse, etc.

1.2 Componenti funzionali

Tutti i calcolatori sono composti da:

- **Memoria**: che utilizziamo per conservare i dati e programmi da utilizzare
- **Unità input/output** per inserire e far uscire dei dati (Es. Tastiera, schermo..)
- **Rete di interconnessione di bus**: è formata dai collegamenti Bus, i fili di interconnessione che collegano il processore sia alla memoria che alle periferie.
- **Processore**: l'unità fondamentale del calcolatore, elabora le istruzioni che avvengono al suo interno, è composto da due unità: Unità aritmetica logica (o ALU) e Unità di controllo.

1.2.1 La memoria

Esistono diversi tipi di memoria:

- **La memoria di massa** è non volatile, ovvero conserva i dati anche senza alimentazione. È poco costosa, ha una grande capacità ma è lenta.
- **Memoria principale RAM dinamica**, creata con diversi transistor. È volatile, cioè perde i dati se non vi è alimentazione, se non sono rilette con la massima corrente possibile ad ogni intervallo di tempo. In media il tempo di risposta per fornire un dato è 20 nanosecondi circa, solitamente presenta 8 transistor. È costosa ma veloce, ha un tempo di accesso costante.
- **Memoria cache o RAM statica**, ha una velocità maggiore rispetto a quella dinamica, ma ha anche un costo maggiore. Non ha bisogno di rinfrescare il dato e lo fornisce in 2 nanosecondi, presenta 2 transistor.

1.2.2 Il processore

Unità aritmetica-logica o ALU: è la parte esecutiva del processore, prende gli operandi e fornisce i risultati attraverso varie operazioni. Utilizza i registri come memoria interna, la dimensione dei registri è uguale alla parola di memoria.

Unità di controllo: governa il coordinamento delle operazioni del processore e ne scandisce il tempo. Deve riconoscere se sono stati eseguiti tutti i passi di un'operazione ed accorciare o allungare il tempo per concluderla se necessario

Il processore esegue le operazioni seguendo queste fasi: Preleva istruzione in memoria -> La codifica -> Governa le azioni successive -> Memorizza l'informazione nei registri

Il flusso di operazioni si interrompe solo quando si verifica un'interrupt.

1.3 Concetti operativi di base

Il processore riconosce 3 tipi di operazioni:

- **Trasferimento dalla memoria al processore**: Load R1, LOC Prende un dato dalla memoria e lo trasporta al processore.
- **Trasferimento dal processore alla memoria**: Str R4, LOC Prende il contenuto del registro del primo operando e lo trasferisce nella locazione di memoria
- **Operazioni aritmetiche**: Add R4,R3,R2 Il primo operando è il registro dove trasferiamo il risultato, gli altri due operandi sono i due registri da sommare.

1.3.1 Connessione fra processore e memoria

Nel processore vi sono alcuni **registri specializzati per alcune funzioni**:

- **PC** (program counter): contiene l'indirizzo della parola di memoria con la prossima istruzione da eseguire
- **IR** (instruction register): contiene l'istruzione corrente e ne permette la decodifica.

1.3.2 Passi dell'istruzione

Un'istruzione per essere eseguita esegue questi passi:

- Il programma (la lista di istruzioni) viene caricato nella memoria principale
- L'istruzione viene prelevata dalla memoria dal PC
- L'istruzione viene trasportata dal PC all'IR. Il PC punta all'istruzione successiva
- Nell'IR l'istruzione viene decodificata
- L'istruzione viene eseguita

1.3.3 Interruzioni

Mentre eseguiamo un'istruzione, può capitare che arrivi un segnale di interruzione, che ferma temporaneamente il programma. Il segnale arriva dalla periferia (dai dispositivi di input) e dice al processore di eseguire la routine di interruzione, che esegue l'istruzione mandata dalla periferia.

Una volta eseguita la routine, il processore riprende l'esecuzione del programma che aveva lasciato in pausa. Questo tipo di segnali fa sì che l'esecuzione di istruzioni non sia deterministica: cioè ogni programma può dipendere anche dai segnali che arrivano dall'esterno.

1.4 Prestazioni di un processore

Per misurare le prestazioni di un processore ci si basa sulla velocità di esecuzione delle istruzioni e dalla tecnologia dello sviluppo delle componenti funzionali.

- Per quanto riguarda la **velocità di esecuzione**, i transistor sono fondamentali: quando leggiamo un dato in un registro, più velocemente viaggia la corrente nel transistor, più velocemente viene eseguita l'istruzione. Questa viene chiamata **velocità di commutazione** e dipende dalle dimensioni di un transistor: più piccolo è, più aumenta la velocità (e si possono inserire più transistor)
- Per quanto riguarda l'organizzazione dell'hardware, troviamo il concetto fondamentale di **parallelismo**:

A livello di **istruzioni**: il parallelismo si basa nell'eseguire più istruzioni contemporaneamente grazie al pipelining

A livello di **core**: un chip può possedere più core, che sono le unità di elaborazione del processore

A livello di **sistema**: esistono calcolatori che hanno più processori che comunicano fra loro ed una memoria condivisa o distinta.

CAPITOLO 2 - ISTRUZIONI MACCHINA

2.1 Introduzione

Un processore è caratterizzato da una serie di istruzioni mappate in linguaggio macchina, le **ISA, instruction set architecture**, che ci indicano:

- Il nome delle istruzioni ed operazioni eseguibili
- I dati che possiamo utilizzare
- Le regole di combinazione dei dati

L'ISA è l'interfaccia tra hardware e software.

2.2 Memoria del calcolatore

La memoria è organizzata in tante celle, capaci di memorizzare un bit. Queste si possono raggruppare in gruppi di n celle, solitamente pari a potenze di 2, chiamate parole di memoria (memory word). La memoria consiste nella successione delle parole di memoria, numerate da 0 a $n-1$, dove n è il vettore di parole, cioè l'insieme di tutte le parole presenti in memoria.

2.2.1 Indirizzamento

Per leggere o scrivere un dato dalla memoria è necessario identificare la parola di memoria tramite un indirizzo, che contiene la locazione di memoria. Se immaginassimo la memoria graficamente, gli indirizzi con numero più basso si troverebbero in alto; quelli con numero più alto si troverebbero in basso. È opportuno distinguere la dimensione (effettiva) di una memoria dal suo spazio di indirizzamento: lo spazio di indirizzamento è il numero massimo di indirizzi possibili della

memoria. Il tipo di architettura (larghezza dei registri) (ad esempio, 32-bit o 64-bit) determina la dimensione dello spazio di indirizzamento. Lo spazio di indirizzamento è uguale a 2^n .

2.2.2 Bus di indirizzi

La rete di interconnessione è formata da bus di dati tra il processore e la memoria; da bus di indirizzi, dove viaggiano gli indirizzi delle locazioni di memoria e da bus di controllo, dove passano i segnali che il processore manda alla memoria e viceversa (read e write).

2.2.3 Indirizzamento a byte

L'indirizzamento a byte nasce per l'esigenza di indirizzare non solo una parola di memoria, ma anche uno specifico byte (ricordiamo 1 byte = 8 bit). Possiamo immaginare di suddividere **la parola di memoria, formata da 32 bit**, in sequenza da 1 byte.

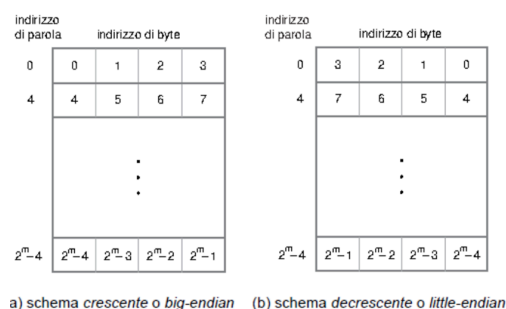
Due parole di memoria consecutive avranno indirizzi che differiscono di 4 (ovvero della lunghezza in byte della parola) Per la parola con indirizzo 0, i byte consecutivi avranno indirizzi 0, 1, 2, 3

Ordinamento a byte Nella memoria indirizzabile per byte, l'ordine dei byte di una parola ha due alternative di indirizzamento:

- Schema crescente (big endian): ordiniamo gli indirizzi partendo da 0 da sinistra verso destra. Presenta il bit più significativo a sinistra, quindi per un byte si ha b6, .. b1, b0
- Schema decrescente (little endian): ordiniamo gli indirizzi partendo da 0 da destra verso sinistra. Il byte più significativo è a destra.

Quando una parola ha indirizzo multiplo della lunghezza della parola in byte lo schema di indirizzamento è allineato.

Lo schema più diffuso è quello decrescente poiché presenta le cifre significative a sinistra.



2.3 Operazioni di memoria

Le operazioni di base che il processore può eseguire con la memoria o viceversa sono due:

- Lettura o read o LOAD: un dato viene prelevato da un indirizzo di memoria e viene scritto in un registro del processore
- Scrittura o write o STORE: un dato viene passato dal processore alla memoria e viene salvato in un indirizzo di memoria

2.4 Istruzioni macchina

Esistono 4 categorie di istruzioni macchina:

- Trasferimento di dati (Load e Store)
- Operazioni algebriche o logiche sui dati
- Controllo della sequenza di esecuzione delle istruzioni
- Trasferimento dati dalle periferiche input al processore o dal processore alle periferiche output

2.4.1 Notazione RTN

La Register transfer notation, notazione RTN, rappresenta istruzioni macchina ed è simile al linguaggio assembly. Utilizza:

- Costanti numeriche o simboliche per indirizzi di parole di memoria
- Parentesi quadrate, che indicano il prelievo del contenuto presente nell'indirizzo
- Nome dei registri
- Freccia verso sinistra, indica la copia di un valore. Il valore a sinistra rappresenta la destinazione, a destra indichiamo la sorgente.

2.4.2 Notazione simbolica

La notazione simbolica rappresenta il linguaggio macchina simbolico, o linguaggio Assemblativo (Assembly)

Ogni istruzione specifica un'operazione da eseguire e gli operandi che utilizziamo. Usiamo quindi:

- Un codice operativo che specifica mediante parole inglesi l'operazione
- Gli operandi di cui il primo rappresenta la destinazione e gli altri la sorgente.

2.5 Architetture RISC e CISC

Vi sono due tipologie di progettazione per realizzare un processore:

- **Architettura RISC**, reduced instruction set computer: Ogni istruzione occupa una parola di memoria, il che semplifica la realizzazione dell'hardware, che occupa meno spazio poiché ha meno transistor. L'accesso agli operandi in memoria avviene solo tramite le istruzioni Load e Store. Tuttavia, lo spazio ridotto che deve occupare un'istruzione vincola la complessità e i tipi di istruzioni disponibili.
- **Architettura CISC**, complex instruction set computer: La lunghezza di bit di ogni istruzione è variabile, il che comporta una costruzione di hardware più complessi con più transistor ed istruzioni molto più complesse. In passato si pensava che le CISC fossero più veloci, in realtà le RISC sono quelle migliori.

2.6 Istruzioni RISC

L'architettura **RISC si basa su una struttura Load/store**: ogni valore è preso dalla memoria e trasferito in un registro, vengono risolte le operazioni aritmetico-logiche e il nuovo valore viene scritto in memoria (no nel CISC).

Queste sono le uniche 2 istruzioni che mi permettono di accedere in memoria o di scrivere qualcosa in memoria: nelle operazioni aritmetico logiche, infatti, i valori vengono presi o dai registri o dall'istruzione stessa.

2.6.1 Formato di istruzioni

- Load d,s La prima variabile è la destinazione, cioè un registro, la seconda è la sorgente, cioè la locazione di memoria.
- Store s, d La prima variabile è la sorgente, il registro da dove preleviamo il valore; la seconda variabile è la destinazione cioè la locazione di memoria
- Add d, S1,S2 Istruzione aritmetica che indica la somma: la prima variabile è la destinazione, ed è sempre un registro, le altre due sono gli operandi da sommare, le sorgenti. A volte la destinazione coincide con il registro della sorgente, quindi il valore presente nel registro sorgente viene sovrascritto e aggiornato dall'operatore.

2.6.3 Esecuzione delle istruzioni

Considerando che nelle architetture RISC ogni istruzione occupa una parola di memoria, assumiamo una lunghezza di 32 bit per capire come si comporta il Program Counter.

Il registro PC, per ogni istruzione successiva, punta all'indirizzo che ha un numero incrementato di 4 byte (appunto 32 bit). Una volta che il PC ha puntato l'indirizzo contenente l'istruzione, avviene il prelievo dell'istruzione da parte dell'Instruction Register, che preleva il contenuto dell'indirizzo, decodifica l'istruzione e la esegue.

2.7 Modi di indirizzamento

Nelle istruzioni esistono diversi modi di indirizzamento (accesso ai dati):

- **Modo di registro**: il dato è contenuto nel registro (Esempio Add R3,R3,R5)
- **Modo assoluto o diretto**: il dato si trova in una variabile che indica l'indirizzo della locazione di memoria che contiene la variabile. (Esempio: Load R2, NUM1)
- **Modo immediato**: specifichiamo all'interno dell'istruzione l'indirizzo della locazione di memoria di una costante, la indichiamo con il carattere # (Esempio: ADD R3,R5,#200)
- **Modo indiretto**: nell'istruzione specifichiamo un registro nel quale c'è l'indirizzo che punta a una parola di memoria, il registro fa quindi da puntatore

2.7.2 Esempio di indirizzamento indiretto

La dereferenziazione $A = *B$, cioè puntare all'indirizzo di una variabile e assegnare il valore contenuto in quell'indirizzo a un'altra variabile, in assembly è scritta così:

LOAD R2, B LOAD R3, (R2) STORE R3, A "Carico l'indirizzo di B in r2, carico il contenuto di r2 (quello che punta r2) in r3, conservo r3 nella locazione di memoria A"

2.7.3 Modo di indirizzamento con indice e spiazzamento (costante)

Esiste anche il modo di indirizzamento con indice e costante, nel caso ci sia bisogno di un valore costante sommato a un registro, per arrivare all'indirizzo al quale si vuole puntare.

LOAD R2, 20(R5) "Punta all'indirizzo ottenuto sommando il valore costante (spiazzamento) con il contenuto del registro (indice) e trasferisci il contenuto dell'indirizzo in R5."

2.7.4 Modo con base e indice (variabile)

Il modo con base e indice è una variante del modo con indice e spiazzamento: il valore prelevato è dato dalla somma di due registri, quindi lo spiazzamento può variare:

LOAD (Ri, Rj)

2.8 Linguaggio assembleativo

Un assembler è un programma che permette di verificare la correttezza delle istruzioni e di codificare in linguaggio macchina, dopo aver verificato che le istruzioni e la loro sintassi siano corrette

2.8.1 Direttive di assembler

Esistono delle direttive di assembler non codificate come istruzioni macchina, forniscono istruzioni all'assembler su come trattare il codice sorgente. Possono avere la funzione di:

- Codificare simboli con dei valori costanti (equate) **VENTI EQU 20** (Ogni volta che incontri il simbolo VENTI sostituisci il valore 20)
- Decidere dove allocare i dati in memoria
- Decidere dove allocare le istruzioni (risultati o operandi) **ORIGIN 100** (ORIGIN alloca le istruzioni dall'indirizzo 100)
- **RESERVE 4** Reserve riserva uno spazio di 4 byte per conservare l'etichetta somma. Le etichette vengono associate dall'assembler a un indirizzo di memoria
- **DATAWORD 150** Dataword riserva una parola di memoria per l'etichetta N e inserisce il valore 150 nella locazione che contiene quest'etichetta

2.8.2 Assemblaggio ed esecuzione

Finito il programma, l'assembler crea un file contenente il programma oggetto, che viene caricato in memoria dal loader quando ne viene richiesta l'esecuzione. Il loader segue solo le direttive utili per sé stesso: ad esempio, carica le istruzioni del programma a partire da dove richiesto con la direttiva ORIGIN. Possiamo anche eseguire il programma in modalità debug, con una istruzione alla volta e se dobbiamo specificare che un valore numerico è scritto in binario o esadecimale, lo scriviamo con i prefissi % e 0x

2.9 Gestione pila

La pila o stack è una struttura dati caratterizzata da una lista di elementi che si possono inserire o prelevare solo dalla testa. L'indirizzo più in fondo è chiamato base, quello più in alto cima della pila. Quindi cresce verso l'alto e decresce verso il basso. È una struttura di tipo LIFO (last in first out): l'ultimo elemento a entrare è quello che estraggo, per questo operiamo solo con la cima, inserendo un nuovo valore (push) o prelevandolo (pop).

2.9.1 Operazione push e pop

Nel processore vi è anche un registro che punta alla pila, ovvero SP (stack pointer), grazie al quale gestiamo push e pop.

Con le operazioni di push e pop non perdiamo i valori precedenti, ma aggiornando la cima della pila, decrementiamo o incrementiamo l'indirizzo a cui punta SP aggiungendo o togliendo dati

- Istruzione **push**: dobbiamo decrementare SP di 4, per poi aggiornare il valore a cui punta SP (la nuova cima) con una STORE: **SUBTRACT SP,SP,#4 STORE Rj, (SP)**
- Istruzione **pop**: preleviamo il contenuto della pila, incrementiamo di 4 SP, aggiornandolo ad una nuova cima **LOAD Rj, (SP) ADD SP,SP,#4**

2.10 Gestione di sottoprogrammi

Una subroutine è una sequenza di istruzioni che svolge un compito specifico e può essere chiamata (o invocata) da diverse parti del programma principale. Quando una subroutine viene chiamata, il flusso del programma salta alla subroutine, esegue le istruzioni al suo interno e poi ritorna al punto di chiamata nel programma principale per continuare l'esecuzione.

CALL SUB Il codice operativo CALL indica la chiamata della funzione e SUB il nome della subroutine. Alla fine della routine inseriamo una return instruction, che ci permette di tornare al programma principale.

2.10.1 Metodo di collegamento

Quando viene letta l'istruzione call, il PC punta alla prima istruzione della subroutine. Per tornare al programma principale e far puntare il PC all'indirizzo dell'istruzione successiva alla Call esiste il registro di collegamento, il Link Register (LR).

Quest'ultimo ci permette di salvare il valore del PC prima della chiamata, in questo modo con la return prendiamo l'indirizzo salvato in LR e lo copiamo in PC, che quindi punta all'istruzione successiva alla call.

2.10.2 Annidamento

Capita di dover inserire delle call all'interno delle subroutine ed eseguire delle chiamate annidate. Il procedimento di chiamata che usiamo è lo stesso con la call nel programma chiamante e chiamato. Si pone però il problema della sovrascrittura del registro LR a causa delle diverse chiamate return che salvano l'indirizzo di PC dell'ultima call. Per evitare questo problema usiamo la pila, per salvare l'ultimo indirizzo di rientro in cima alla pila (SP) e ritornare al programma principale con gli indirizzi corretti, poiché l'indirizzo di rientro è l'ultimo generato. Diciamo quindi che il meccanismo di chiamata si serve della pila in modo implicito.

2.10.3 Passaggio di parametri

Esistono 2 modi per passare dei parametri alla subroutine:

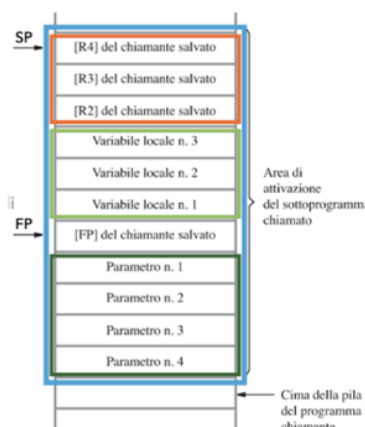
- **Tramite registri:** il programma chiamante passa valori a dei registri che devono essere letti nel programma chiamato. Questo metodo è veloce, dato che i registri sono la memoria più veloce di un elaboratore, ma possono essere limitati per tutte le istruzioni della subroutine.
- **Tramite pila:** il chiamante passa nella pila un qualsiasi numero di valori, nelle giuste posizioni, che il chiamato legge. È un processo più lento perché il processore opera con la memoria.

Il passaggio di parametri deve avvenire in un unico modo, sia nel chiamante che nel chiamato.

2.10.4 Area di attivazione

Quando passiamo i parametri tramite stack i dati vengono impilati seguendo un ordine, che costituisce l'area di attivazione, activation record. Gli strati di quest'area/pila a partire dalla base sono composti da:

- I parametri da passare, quindi SP punta alla cima della pila.
- Dopo aver passato i parametri viene inserito il registro FP (Frame Pointer)
- Sopra il registro FP impiliamo le variabili locali e temporanee
- Infine impiliamo i registri da salvare che a fine routine devono essere ripristinati



Registro FP

Per prelevare i parametri, la routine deve contare quante parole di memoria si trovano al di sotto di SP. Per questo si usa il registro FP: è il puntatore all'area di attivazione e punta subito dopo alla chiamata al sottoprogramma dal chiamante, per

avere un registro in una posizione che ci permette con un incremento positivo di puntare ai parametri, ed uno negativo alle variabili locali.

- Il punto in cui FP viene salvato è dato dalla parola di memoria che punta al vecchio FP del chiamante.

Quindi la routine impila FP dopo che il chiamante ha passato i parametri, aggiornando la cima della pila con SP, poi impilando le eventuali variabili locali e registri. Quando bisogna ripristinare i valori di registri e del registro FP la routine li ripristina e i parametri vengono ripristinati dal chiamante.

2.11 Ulteriori istruzioni macchina

AND, OR, NOT sono operazioni logiche.

And R4,R2,R3 In questo caso avremo un prodotto bit a bit, cioè facciamo il **prodotto** di ogni singolo bit di entrambi i registri delle rispettive posizioni.

And R2, R2, #0xFF L'istruzione AND R2, R2, #0xFF :

- Effettua un'operazione di AND bit a bit tra il valore già presente nel registro R2 e l'immediato #0xFF (che rappresenta il valore binario 00000000 00000000 00000000 11111111).
- Dopo l'operazione, i primi 24 bit di R2 diventano 0. Gli ultimi 8 bit di R2 rimangono invariati rispetto al valore originale di R2.

2.11.1 Scorrimento

Lo scorrimento (shift) scorre i bit di un certo numero di posizioni verso destra o verso sinistra.

- LshiftL **LSL** Ri, Rj, #contatore

Logic shift left, scorrimento verso sinistra di Rj di "contatore" posizioni; il risultato viene copiato in Ri. I bit più significativi vengono prima posti uno a uno nel bit di riporto (carry) e poi eliminati. Le posizioni a destra vengono riempite con 0.

Se scorriamo di una sola posizione verso sinistra, ciò che otteniamo è il numero raddoppiato di 2: posizioniamo i bit

in modo che si trovi con una potenza di 2 in più. Quindi è una moltiplicazione di 2



- LshiftR **LSR** Ri, Rj, #contatore

Logic shift right, si perdono i bit meno significativi a destra e le posizioni vuote a sinistra vengono riempite con 0. Il bit di carry viene aggiornato in base allo scorrimento.

Se scorriamo a destra di una posizione si ottiene il numero divisione per 2: posizioniamo i bit in una potenza di 2 in meno.

Quindi è una divisione senza segno di 2. Se il bit di carry (il resto del numero) è 0, allora il numero è pari, se è 1, è dispari.



- AShiftR

In un'operazione di shift aritmetico a destra, il bit più significativo (MSB, Most Significant Bit), che rappresenta il segno in una rappresentazione in complemento a due, viene **replicato** in ogni posizione liberata dallo shift. È una divisione con segno di 2.



- Impaccamento: Se un dato è contenuto in pochi bit, ci conviene usare un unico byte per contenerlo. Facciamo quindi uno scorrimento verso sinistra di 4 bit e poi un OR con il dato che contiene a destra i bit meno significativi.

2.11.2 Rotazione

Le istruzioni di rotazione fanno in modo che non si elimini nessun bit, ma si effettui una rotazione che faccia scorrere i bit dall'estremo destro al sinistro o viceversa.

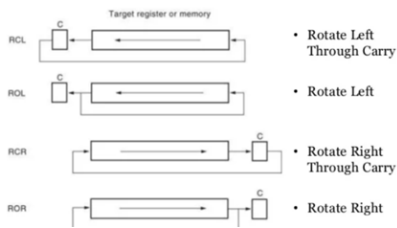
- Rotazione verso sinistra: **RotateL R3, R3, #2**

La rotazione scorre 2 bit da sinistra, li copia prima sul bit di riporto, e poi li copia come ultimi due bit a destra al rientro.

- Rotazione verso destra: **RotateR R3, R3, #2**

La rotazione scorre 2 bit da destra, li copia prima sul bit di riporto e poi li copia come primi due bit a sinistra al rientro.

- Oppure, la rotazione può coinvolgere il bit di riporto che si aggiunge come posizione ulteriore di rotazione: **RotateLC R3, #2**



2.11.3 Moltiplicazione e Divisione

Vi sono anche delle istruzioni in RISC per la moltiplicazione e divisione

- **Moltiplicazione: Multiply Rk,Ri,Rj** Ri ed Rj sono i registri da moltiplicare, Rk è la destinazione. Spesso il prodotto non può essere inserito nello spazio di n bit dei fattori; per risolvere questo problema il compilatore suddivide il risultato in tal modo:
 - La parte di bit meno significativa vanno nel registro di destinazione Rk La parte di bit più significativa vanno nel registro successivo, "Rk+1"
- **Divisione: Divide Rk,Ri,Rj** Dobbiamo fare attenzione all'ordine dei registri sorgente, poiché il primo a sinistra (Rj) è il numeratore, Ri è il denominatore. Non ci importa il numero di bit che ci occorre per allocare il risultato. L'eventuale resto viene salvato nel registro successivo a quello di destinazione Rk+1

2.12 Valori immediati a 32 bit

Nel RISC ogni registro è grande 1 parola di memoria (32 bit), quando però passiamo valori immediati attraverso i modi di indirizzamento immediato e assoluto, occuperanno solo 16 bit, perchè si lascia spazio per l'intera istruzione. Esiste un meccanismo del processore che estende a 0 i 16 bit più significativi riempiendo solo quelli meno significativi. Il problema si pone quando dobbiamo codificare valori immediati più grandi di 16 bit. Quindi usiamo le istruzioni:

- **OrHigh R2,R0,#0x2000** OrHigh si serve dell'OR per i bit più significativi, quindi pone a 0 i bit meno significativi e alloca le prime 4 cifre del valore immediato, che occupano 16 bit nei bit più significativi. Il registro R0 contiene tutti 0, quindi un OR tra 0 e 0 restituisce 0 nei bit meno significativi, l'OR logico tra 0 e il valore immediato lascia il valore invariato, allocandolo nei bit più significativi del registro R2
- **Or R2,R2,#0x4FF0** Or usa solo i bit meno significativi: avendo già allocato le prime 4 cifre del valore e avendo occupato quindi i primi 16 bit più significativi, facciamo l'OR logico tra il valore già allocato e la parte del valore non allocato; quindi, allochiamo **#0x4FF0** nei bit meno significativi, occupando gli ultimi 16 bit.

2.13 Insieme di istruzioni CISC

In un'architettura CISC possiamo utilizzare diverse parole di memoria per una singola istruzione; quindi, non siamo vincolati da architettura LOAD/STORE: effettuiamo operazioni aritmetico-logiche su operandi che si trovano nei registri.

- **Operazione di addizione: ADD B,A** Abbiamo solo 2 operandi e un codice operativo, e, l'operando B rappresenta sia la sorgente che la destinazione della somma. I due operandi inoltre sono le locazioni di memoria.
- **Istruzione move: MOVE C,B ADD C,A** Usiamo 2 operandi, il primo è la destinazione, il secondo la sorgente, entrambe locazioni di memoria. Sostituisce la LOAD/STORE usata in RISC.

2.13.1 Istruzioni di autoincremento e autodecremento

Per la pila, nei RISC riservavamo uno spazio di byte congruo per tutti i valori e poi li inserivamo e toglievamo con push e pop. In CISC abbiamo l'autodecremento e autoincremento

- **Istruzione di autodecremento (push): MOVE -(SP), ELEMENTO** Decrementiamo l'indirizzo puntato da SP di un certo numero di byte, in base alla dimensione del dato, e inserire in cima alla pila il valore "ELEMENTO"
- **Istruzione di autoincremento (pop): MOVE ELEMENTO, (SP)+** Incrementiamo l'indirizzo puntato da SP di un certo numero di byte, in base alla dimensione del dato, estraendo l'elemento in cima alla pila.

2.13.2 Indirizzamento con modo relativo a PC

Nei processori RISC non si può indicare il registro PC, mentre nei CISC si può utilizzare per l'indirizzamento con indice e spiazzamento: X(PC) In tal caso indicheremo una parola di memoria traslata di X byte rispetto a PC

2.13.3 Bit di esito o condizione

Quando eseguiamo operazioni logico-aritmetiche, il processore tiene conto dei bit di esito:

- **N (negativo):** vale 1 se l'operazione restituisce un numero negativo. 0 se restituisce un numero positivo
- **Z (zero):** vale 1 se il risultato dell'operazione restituisce zero, altrimenti vale 0
- **V (overflow) (trabocco):** vale 1 se vi è stato trabocco nell'operazione, 0 se non c'è stato
- **C (carry - riporto):** vale 1 se c'è riporto, 0 se non c'è

2.14 Stili RISC e CISC

- **RISC:** -Molte istruzioni ma semplici -Ogni istruzione occupa una parola di memoria -Esecuzione rapida -Hardware semplice -Architettura LOAD/STORE -Possiamo effettuare operazioni aritmetico-logiche solamente tramite registri e valori immediati.
- **CISC:** -Poche istruzioni complesse -Ogni istruzione può occupare più di una parola di memoria -Esecuzione lenta -Hardware complesso -Possiamo effettuare operazioni aritmetico-logiche usando operando in memoria e registri del processore.

Fino a pochi anni fa si usavano di più i CISC perchè si potevano usare meno istruzioni e quindi meno memoria. Dato che oggi non c'è più questo problema, ora si usano di più i RISC perchè hanno più vantaggi.

2.15 Codifica di istruzioni

****Istruzioni con 3 registri e codice operativo (es. ADD, SUB, etc.):**

- **Registro:** Se il processore dispone di 32 registri, ogni registro sarà codificato con **5 bit** (poiché $2^5 = 32$).
- **Registri coinvolti:** Con 3 registri, si occupano **15 bit** ($5 \times 3 = 15$).
- **Codice operativo:** I restanti **17 bit** ($32 - 15 = 17$) sono dedicati al **codice operativo**.
- **Posizione dei registri:** La posizione dei campi per i registri all'interno dell'istruzione è definita dall'architettura del processore.

Istruzioni di salto:

- Anche le istruzioni di salto usano questa codifica.
- Il **valore immediato** specificato in queste istruzioni rappresenta lo **spiazzamento** (offset) tra l'etichetta di destinazione e l'indirizzo corrente del programma.

2. Istruzioni di chiamata a sottoprogramma (es. CALL, JUMP, etc.):

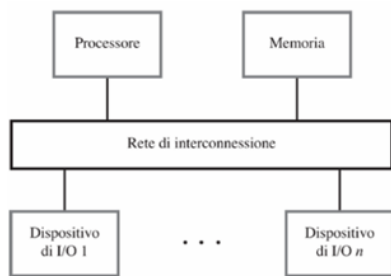
- **Codice operativo:** In queste istruzioni, il codice operativo è codificato in **6 bit**.
- **Valore immediato:** I rimanenti **26 bit** ($32 - 6 = 26$) rappresentano un **valore immediato** che potrebbe contenere:
 - Un indirizzo assoluto.
 - Un indirizzo relativo.
 - Un'etichetta o altro, in base all'architettura specifica.

CAPITOLO 3 - OPERAZIONI DI INGRESSO/USCITA

3.1 Accesso a dispositivi di I/O

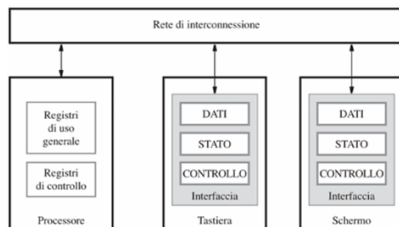
Tutte le componenti del calcolatore sono collegate tramite fili di bus a una rete di interconnessione, tra questi vi sono anche i dispositivi di I/O, dette periferiche. Ci sono due modi per accedere ad esse.

1. **Creazione di apposite istruzioni per indirizzare le periferiche**
2. **Unificazione degli indirizzi di memoria:** vi sono degli indirizzi speciali salvati nello **spazio di indirizzamento del processore**, associati a una determinata periferica; è il metodo più utilizzato.



3.1.1 Interfaccia dei dispositivi

Facendo riferimento anche all'unificazione degli indirizzi di memoria, lo spazio di indirizzamento non è più 2, perchè vi sono indirizzi riservati alle periferiche. I dispositivi di I/O sono collegati alla rete di interconnessione tramite un circuito, detto interfaccia del dispositivo, dove sono contenuti i registri **dati**, dello **stato corrente** e delle **informazioni di controllo**. L'accesso ai registri avviene mediante locazioni di memoria.



3.1.2 I/O controllati da programma

Dato che le periferiche forniscono dati più lentamente, bisogna trovare una sincronizzazione tra il processore e la periferica. Prendiamo ad esempio la tastiera per input ed il video per output.

- **Tastiera:** Ogni tasto/ogni carattere corrisponde a un codice binario. Quest'ultimo viene salvato nel registro dati dell'interfaccia della tastiera, che fa da buffer. Viene impostato dalla tastiera un bit di stato, per dire al processore di aver ricevuto un dato. (continua per spiegazione dettagliata)
- **Video:** Avviene lo stesso procedimento: quando un dato è stampato a video, viene impostato il bit di stato che dice che il processore può ricevere un altro dato.

3.1.3 Lettura dati da tastiera

Registri contenuti nell'interfaccia della tastiera:

- Registro dati: KBD_DATA (kbd = keyboard)
- Registro di stato: KBD_STATUS
- Registro di controllo: KBD_CONT (control)

KBD_DATA occupa uno spazio di 8 bit, corrispondente a un carattere di codifica ASCII, cioè al dato che dovrà contenere. KBD_STATUS presenta un bit di stato "KIN", che viene posto a 1 se pronto a ricevere un altro dato, mentre viene posto a 0

Indirizzo	7	6	5	4	3	2	1	0	
0x4000									KBD_DATA
0x4004							KIN	KIRQ	KBD_STATUS
0x4008							KIE		KBD_CONT

se si è letto un dato da KBD_DATA

3.1.4 Scrittura caratteri sul video

- Registro dati: DISP_DATA (DISP = DISPLAY). Anche questo occupa 8 bit corrispondente ad un carattere codificato in ASCII
- Registro di stato: DISP_STATUS
- Registro di controllo: DISP_CONT (control)

Anche DISP_DATA occupa 8 bit = carattere codificato in ASCII. DISP_STATUS presenta il bit di stato DOUT, posto a 1 se pronto a ricevere un altro carattere da DISP_DATA, posto a 0 se ha scritto un dato da DISP_DATA.

	7	6	5	4	3	2	1	0	
0x4010									DISP_DATA
0x4014							DOUT	DIRQ	DISP_STATUS
0x4018							DIE		DISP_CONT

3.2 Interruzioni

Le **interruzioni** rappresentano un meccanismo fondamentale per la comunicazione tra periferiche e processore, progettato per evitare che il processore sprechi tempo in **cicli di attesa**. Questi cicli si verificano quando il processore verifica continuamente il bit che indica la disponibilità del dato o la possibilità di inviarlo alla periferica. Le interruzioni risolvono questo problema in modo efficiente.

Quando una periferica necessita di interagire con il processore, invia un **segnale di interruzione** tramite una linea dedicata del bus di controllo, chiamata **INT_REQ** (Interrupt Request) o **IRQ**. Ponendo questa linea ad **1** (stato alto), la periferica informa il processore di una richiesta.

A questo punto, il processore:

1. **Interrompe** l'esecuzione del programma corrente.
2. **Salta** all'indirizzo iniziale della **routine di servizio dell'interruzione** (Interrupt Service Routine, ISR), un programma specifico per gestire l'evento segnalato dalla periferica.
3. Dopo aver completato l'ISR, **riprende** l'esecuzione del programma principale dal punto esatto in cui era stato interrotto.

Grazie a questo meccanismo, il processore può svolgere altre attività mentre attende richieste dalle periferiche. Legge o scrive i dati nel **registro dati buffer** dell'interfaccia solo quando richiesto, eliminando sprechi di tempo e risorse.

3.2.2 Routine di servizio dell'interruzione

La routine di servizio di interruzione ha caratteristiche simili alla chiamata a sottoprogramma:

- Quando il processore riceve l'interrupt request, il PC punta alla routine di servizio e deve essere salvato nella pila o su un registro per riprendere la compilazione del programma.
- Finita la routine di servizio, si torna al programma principale con l'istruzione return-from-interrupt, che riporta PC all'indirizzo successivo all'istruzione che stava eseguendo
- Il processore informa la periferica dell'avvenuta ricezione dell'interrupt tramite la linea INT_ACK nel bus di controllo.

3.2.3 Servizio delle interruzioni

Un altro registro importante da salvare è il PS (program status), che contiene i bit di stato.

3.2.4 Controllo delle interruzioni

Quando si deve saltare da una routine di servizio ad un'altra, si provocherebbero diversi annidamenti di PS. Per questo vi sono meccanismi per abilitare/disabilitare le richieste di interruzioni:

- Nel processore, nel registro PS, il **bit IE** (Interrupt Enable): quando vale 1 abilita le routine di servizio e quando vale 0 le disabilita.
- Nell'interfaccia periferica, nel registro di controllo il **bit IE** quando vale 1, il dispositivo di I/O è abilitato a dare richieste di interruzioni, quando vale 0 è disabilitato a farlo.

Sequenza di eventi relativi a una richiesta di interruzione:

1. La periferica attiva la richiesta di interruzione (IRQ va alto)
2. Il processore interrompe il programma e salva i registri PC e PS che contiene il bit IE
3. Le interruzioni vengono disabilitate ponendo il bit IE a 0, così da non ricevere ulteriori interruzioni
4. La routine ISR va in esecuzione, la periferica viene informata e IRQ va in basso
5. Il return-from-interrupt ripristina PC e PS quindi anche il bit IE viene ripristinato a 1, si riabilitano le interruzioni e si rientra al programma interrotto

3.2.5 Dispositivi multipli

Quando più dispositivi di I/O sono collegati al processore, possono sorgere due problemi principali con le **interruzioni**:

1. **Identificare il dispositivo** che ha emesso la richiesta.
2. **Gestire richieste simultanee** da dispositivi diversi.

Per risolvere il problema dell'identificazione, si utilizzano due approcci principali:

Polling o scansione

Quando il bit **IRQ** del registro di stato è impostato a 1, il processore **scansiona le periferiche** per determinare quale dispositivo ha emesso la richiesta. Questa tecnica è semplice, ma meno efficiente, poiché richiede al processore di controllare ogni dispositivo in modo sequenziale.

Interruzioni vettorizzate

In questo approccio, la periferica che genera l'interruzione invia un **codice identificativo** (di pochi bit) al processore insieme al segnale di interrupt request. Il processore utilizza questo codice come **indice** nella **tabella dei vettori di interruzione**, che associa ogni codice a un indirizzo specifico della routine di servizio.

- Una volta trovato l'indirizzo della **ISR specifica**, il processore:
 1. Lo carica nel **Program Counter (PC)**.
 2. Esegue direttamente la routine di servizio appropriata.

3.2.6 Annidamento delle interruzioni

Ogni periferica è associata a un **livello di priorità**. Questo livello viene confrontato con quello corrente del processore (salvato in un registro, come il registro PS - **Processor Status**).

- Se una periferica con una priorità più alta richiede un'interruzione, il processore sospende l'esecuzione corrente per servire questa nuova richiesta.
- Durante il servizio della periferica, il livello di priorità del processore si alza per uguagliare quello della periferica in servizio. Questo meccanismo previene che periferiche di priorità inferiore interferiscano.

Quando si serve un'interruzione:

1. Il processore salva il contesto corrente, inclusi i registri e lo stato del programma.
2. Passa all'esecuzione della routine di servizio (ISR - **Interrupt Service Routine**).
3. Al termine della routine, il contesto viene ripristinato, incluso il livello di priorità e il registro PS.

Le interruzioni non eseguite vengono salvate nel registro IPENDING.

Gestione delle interruzioni (interrupt handler) Il registro **IPENDING** tiene traccia delle interruzioni attive e non ancora servite. Ogni bit del registro rappresenta una periferica:

- Se un bit è impostato a 1, significa che c'è una richiesta di interruzione pendente per quella periferica.
- Quando il processore è libero, controlla il registro IPENDING e serve la richiesta attiva con la priorità più alta. Questo meccanismo consente di gestire più richieste simultanee e mantenere una coda logica delle periferiche in attesa.

3.2.7 Richieste di interruzioni simultanee

Quando due o più richieste arrivano simultaneamente, il processore decide quale richiesta arriva per prima tramite uno dei due meccanismi seguenti:

- Scandendo il registro delle periferiche l'ordine di scansione indica il livello di priorità della periferica.
- Usando le interruzioni vettorizzate, che selezionano una periferica per mandare il vettore di interruzione: quando viene alzato il segnale di interrupt request dalla periferica, essa è collegata tramite il bus anche ai circuiti di arbitraggio, che disabilitano la possibilità che altre periferiche possano richiedere ulteriori interruzioni.

3.2.8 Controllo della richiesta

Riassumendo ciò che abbiamo detto: l'abilitazione/disabilitazione delle richiesta può essere fatta dal processore o dall'interfaccia della periferica: di conseguenza, se il processore vuole disabilitare l'interruzione pone il bit IE a 0. Quando invece c'è una richiesta si controlla se il bit di stato del dato e IE sono entrambi a 1, in modo tale che il bit IRQ possa alzare il segnale e abilitare la richiesta di interruzione.

3.2.9 Registri di controllo del processore

- Il registro PS include il bit di abilitazione delle interruzioni IE
- Il registro IPS è usato per salvare automaticamente i contenuti di PS quando una richiesta di interruzione è ricevuta e accettata. Se sono permesse interruzioni annidate, bisogna salvare in pila il contenuto di IPS.
- Il registro IENABLE permette di assegnare un bit a ciascun dispositivo, e quando un bit è 1 il processore accetta le richieste dal dispositivo corrispondente.

- Il registro IPENDING indica le richieste di interruzioni attive.
- I registri sono a 32 bit, quindi possono essere gestite fino a 32 periferiche. L'accesso a tali registri avviene tramite istruzioni speciali, come MoveControl.

3.3 Eccezioni

Qualsiasi evento che provochi un'interruzione è chiamato eccezione:

- Eccezioni da memoria per ripristino da errore: la memoria manda un'interruzione se ci sono dati alterati da segnalare al processore
- Altre eccezioni: tentativo di eseguire istruzioni con codice operativo inesistente; errore per esecuzione di divisione per zero; tentativo di accesso a parti protette della memoria. Se l'eccezione è un errore, il processore rinuncia all'esecuzione del programma principale.

CAPITOLO 4 - STRUTTURA DI BASE DEL PROCESSORE

4.1 Componenti hardware

Le componenti hardware del processore che permettono di eseguire l'istruzione sono:

- Banco registri: fornisce registri nelle operazioni di caricamento in memoria e scrittura in memoria.
- ALU: esegue le operazioni aritmetico/logiche.
- Circuiti di controllo: unità che genera i segnali di controllo.
- Registro IR: contiene l'istruzione da eseguire
- Registro PC: contiene l'indirizzo della prossima istruzione da eseguire
- Generatore di indirizzi: aggiorna PC dopo aver prelevato l'istruzione
- Interfaccia tra processore e memoria: trasferisce i dati dal processore alla memoria

4.1.2 Hardware per l'elaborazione dati

Con le componenti hardware disponibili, è possibile costruire un circuito combinatorio composto da diverse porte logiche. Questo circuito è caratterizzato da:

- **Tempo di propagazione:** il tempo necessario affinché un segnale si propaghi dall'ingresso all'uscita del circuito. È dato dalla somma dei tempi di propagazione delle singole porte logiche presenti nel circuito.
- **Periodo di clock:** l'intervallo di tempo tra due fronti di salita consecutivi di un segnale di clock.
 - Il segnale di clock è un'onda periodica, solitamente quadra, generata da un oscillatore.
 - Il periodo di clock è inversamente proporzionale alla frequenza di clock, cioè al numero di cicli di clock per secondo.
 - Una frequenza di clock maggiore corrisponde a un periodo di clock minore.
- **Flip-Flop:** i registri sono composti da flip-flop, che memorizzano i dati al fronte di salita del clock.

Per ottimizzare le prestazioni, il circuito viene suddiviso in più sezioni, dette **stadi in cascata**, che operano in modo sequenziale. Questa suddivisione permette di ridurre il periodo di clock e aumentare la frequenza di clock, garantendo un'elaborazione più veloce.

4.1.3 Struttura hardware a più stadi

Nei circuiti hardware a più stadi, esistono registri di **"inizio"** e di **"fine"**. Il registro di inizio riceve i valori di input nel primo stadio, mentre il registro di fine memorizza i valori provenienti dall'uscita del circuito nell'ultimo stadio.

Con **n stadi in cascata**, l'esecuzione di un'istruzione viene completata dopo **n cicli di clock**, poiché ogni stadio contribuisce al processo in sequenza.

4.2 Esecuzione di istruzioni

Nel processore RISC per completare ogni qualsiasi istruzione si procede in 5 passi, ognuno corrispondente ad uno stadio hardware.

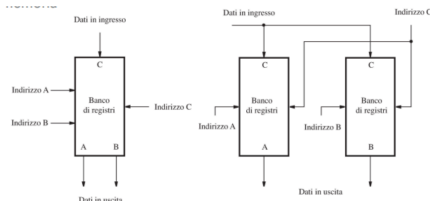
4.3 Hardware e percorso dati

4.3.1 Banco di registri

Il **banco dei registri** è l'unità di memoria che fornisce i registri. Riceve in ingresso due indirizzi, **A** e **B**, dal registro **IR**. Questi indirizzi selezionano simultaneamente due registri sorgente nel banco dei registri, i cui valori vengono forniti sulle uscite **A** e **B**. Inoltre, l'indirizzo **C** seleziona il registro di destinazione per il dato che entra nel banco dei registri.

Tuttavia, quando inviamo due indirizzi al banco dei registri, quest'ultimo deve restituire simultaneamente i due registri sorgente nelle uscite. Per risolvere questo problema, abbiamo due opzioni:

1. **Duplicare la memoria:** Si utilizzano due banchi registri separati. L'indirizzo **A** viene inviato al primo banco, che fornisce il primo registro sorgente in uscita, mentre l'indirizzo **B** viene inviato al secondo banco, che fornisce il secondo registro. L'indirizzo **C**, per il registro di destinazione, viene duplicato in modo da essere disponibile per entrambe le uscite.
2. **Duplicare i circuiti di accesso:** Si utilizza un solo banco dei registri, ma con circuiti duplicati. Ogni circuito riceve in ingresso uno degli indirizzi (**A** o **B**) e fornisce il registro corrispondente come uscita.

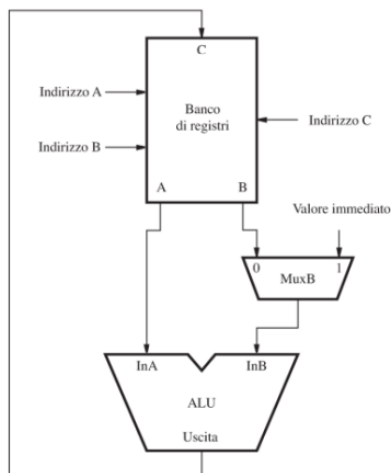


4.3.2 ALU

L'**ALU** (Unità Aritmetico Logica) ha due ingressi collegati alle uscite del **banco dei registri**:

- **Ingresso InA** dell'ALU è collegato all'uscita **A** del banco dei registri.
- **Ingresso InB** dell'ALU riceve il valore tramite un **multiplexer (MuxB)**. Se l'operando è un valore immediato, codificato con la linea di ingresso **"1"**, il **MuxB** seleziona come uscita l'operando immediato. Se l'istruzione prevede due registri come operandi, il **MuxB** sceglie la linea di ingresso **"0"**, poiché l'uscita **B** del banco dei registri è collegata all'ingresso **0** del **MuxB**.

L'uscita dei dati dell'ALU è collegata all'indirizzo **C**, che seleziona il registro di **destinazione** per memorizzare il risultato dell'operazione.



4.3.3 Struttura a 5 stadi

Ogni stadio è completato in un periodo di clock.

1. Prelievo dell'istruzione dalla memoria e incremento di PC
2. Decodifica dell'istruzione e lettura di registri dal banco registri
3. Elaborazione con l'ALU
4. Lettura o scrittura in memoria, se l'istruzione coinvolge un operando in memoria
5. Scrittura nel registro destinazione, se l'istruzione prevede la scrittura in un registro del processore.

- **Operazione Store:** Quando si effettua una scrittura in memoria, il registro interstadio **RM** viene attivato. **RM** prende il dato dal banco dei registri (dall'uscita **RB**) e lo invia alla memoria. Il dato passa da **RB** a **RM** negli stadi 2 e 3, e nello stadio 4 viene inviato alla memoria. Nello stadio 5 non succede nulla.

Passo 5: Il risultato finale dell'operazione viene scritto in un registro, se necessario. Per un'operazione aritmetica, il risultato viene scritto in un registro. Per un'operazione **store**, non avviene scrittura nei registri.

4.3.5 Sezione di prelievo delle istruzioni

Nel **primo stadio**, durante il prelievo dell'istruzione, il **generatore di indirizzi** fornisce l'indirizzo del **PC** (Program Counter), che viene inviato al **multiplexer MuxMa**. Il **MuxMa** seleziona la sorgente da inviare in memoria. In questo stadio, la linea di ingresso del **MuxMa** è sempre impostata su **1**, in modo da inviare l'indirizzo del **PC** alla memoria.

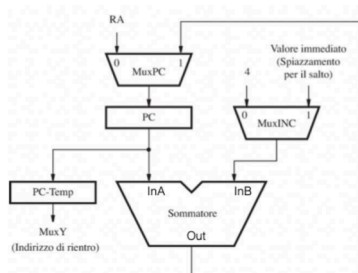
Nel **quarto stadio**, se l'operazione è una **load**, il **MuxMa** seleziona la linea di ingresso **0** e preleva il dato contenuto nell'indirizzo di memoria specificato in **RZ**.

Se l'istruzione in **IR** (Instruction Register) contiene un valore immediato, che è composto da **16 bit**, considerando che l'istruzione deve essere lunga **32 bit**, i **bit più significativi** vengono estesi a **0** tramite un **blocco hardware "immediato"**.

Generatore di indirizzi delle istruzioni

Il **generatore di indirizzi** modifica l'indirizzo contenuto nel Program Counter. Analizziamo i due casi principali:

1. **Operazione senza salto:** Se l'istruzione non è un salto, il contenuto di **PC** viene copiato in un registro interstadio, chiamato **PC-Temp**. All'interno del generatore di indirizzi, un **sommatore** prende in input il valore di **PC** tramite l'ingresso **InA**. Successivamente, tramite il **multiplexer MuxINC**, viene scelto un incremento da sommare a **PC**. La linea di ingresso selezionata è **"0"**, che contiene il valore **4** (cioè i 4 byte che indicano la parola di memoria successiva). Questo valore viene passato all'ingresso **InB** del sommatore. Il risultato della somma **PC + 4** viene inviato al **MuxPC**, che seleziona la linea di ingresso **1** per aggiornare il valore di **PC**.
2. **Operazione di salto:** Se l'operazione è un salto, un **valore immediato** viene sommato a **PC** per ottenere lo spiazzamento dell'indirizzo di salto. In questo caso, il **MuxINC** seleziona la linea di ingresso **"1"**, che contiene il valore dello spiazzamento. Le azioni successive sono simili al caso precedente: il sommatore calcola **PC + spiazzamento**, e il risultato viene inviato al **MuxPC**, che aggiorna **PC** con il nuovo indirizzo.

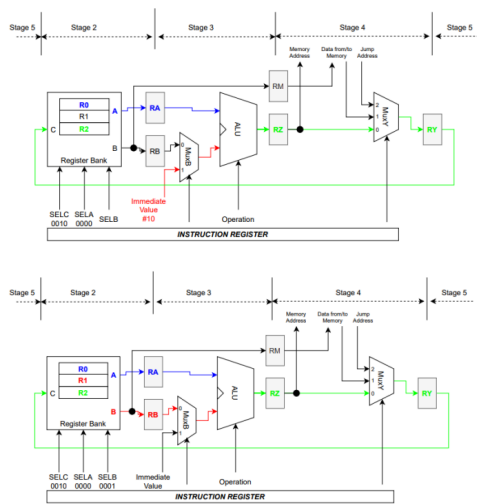


Analizziamo il **data path** per la chiamata a sottoprogramma:

- **CALL SUB1 → CALL_REGISTER R9:** L'istruzione specifica che il registro **R9** contiene l'indirizzo della sottoprocedura da chiamare.
- Una volta che l'istruzione è decodificata, il valore del salto (ovvero l'indirizzo della subroutine) viene memorizzato nel registro **RA**.
- **Aggiornamento di PC:** Dopo la decodifica dell'istruzione di chiamata, il **generatore di indirizzi** viene utilizzato per aggiornare il **PC**. Nel secondo passo, il valore dell'indirizzo della subroutine, contenuto in **RA**, viene inviato al **MuxPC**. La linea di ingresso scelta per il **MuxPC** è **0**, e il valore di **RA** viene utilizzato per aggiornare **PC** con l'indirizzo della subroutine.
- **PC-temp:** Il registro **PC-temp** non viene aggiornato durante la chiamata. Esso contiene l'indirizzo dell'istruzione precedente alla chiamata e verrà utilizzato successivamente come indirizzo per l'istruzione di rientro. **PC-temp** è collegato al **MuxY**, il cui ingresso **2** riceve l'indirizzo di rientro. Questo indirizzo viene poi salvato nel registro interstadio **RY**.

Passi di esecuzione del datapath

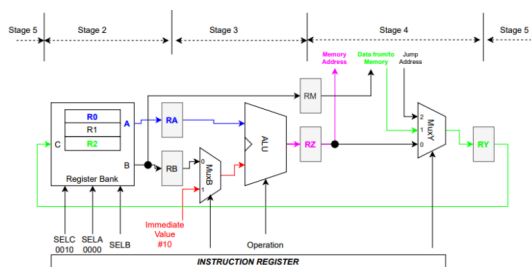
4.4.1 ADD



ADD R3, R4, R5

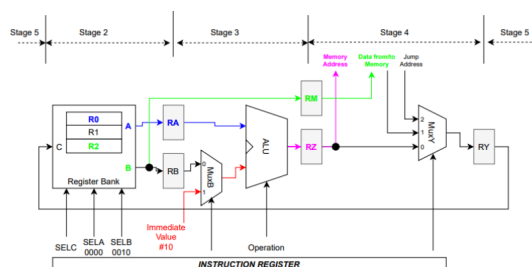
- 1. Prelievo dell'istruzione:** Il PC viene copiato nell'indirizzo di memoria. Si legge l'istruzione dalla memoria e si copia nei dati del IR (Instruction Register). Il PC viene incrementato di 4 per puntare alla successiva istruzione.
- 2. Decodifica dell'istruzione:** L'istruzione viene decodificata. Tramite l'indirizzo A, il valore di R4 viene copiato nel registro RA. Tramite l'indirizzo B, il valore di R5 viene copiato nel registro RB.
- 3. Esecuzione dell'operazione:** Il valore di RA viene passato all'ingresso InA dell'ALU. RB, essendo un registro, viene scelto dal MuxB tramite la linea di ingresso "0" e mandato all'ingresso InB dell'ALU. L'ALU esegue l'operazione di somma sui registri RA e RB. Il risultato della somma viene conservato nel registro RZ.
- 4. Memorizzazione del risultato:** Il valore di RZ passa attraverso la linea "0" di MuxY e viene copiato nel registro intermedio RY.
- 5. Scrittura del risultato:** Il valore di RY viene copiato nel registro R3, utilizzando l'indirizzo C.

4.4.2 Load e Store



Load R5, (X)R7

- 1. Prelievo dell'istruzione:** Il PC viene copiato nell'indirizzo di memoria. Si legge l'istruzione dalla memoria e si copia nei dati del IR (Instruction Register). Il PC viene incrementato di 4 per puntare alla successiva istruzione.
- 2. Decodifica dell'istruzione:** L'istruzione viene decodificata. Tramite l'indirizzo A, il valore di R7 viene copiato nel registro RA.
- 3. Esecuzione dell'operazione:** Il valore di RA viene passato all'ingresso InA dell'ALU. Il valore immediato X passa dalla linea di ingresso "1" del MuxB e poi viene inviato all'ingresso InB dell'ALU. L'ALU esegue la somma tra RA e X. Il risultato della somma viene conservato nel registro RZ.
- 4. Memorizzazione del risultato:** Il valore di RZ passa attraverso il bus degli indirizzi. Vengono letti i dati dalla memoria corrispondente a quell'indirizzo. I dati tornano dalla memoria al MuxY tramite la linea di ingresso "1" e vengono copiati nel registro intermedio RY.
- 5. Scrittura del risultato:** Il valore di RY viene copiato nel registro R5.



Store R6, (X)R8

1. **Prelievo dell'istruzione:** Il **PC** viene copiato nell'indirizzo di memoria. Si legge l'istruzione dalla memoria e si copia nei dati del **IR** (Instruction Register). Il **PC** viene incrementato di 4 per puntare alla successiva istruzione.
2. **Decodifica dell'istruzione:** L'istruzione viene decodificata. Il valore di **R8** viene copiato nel registro **RA**. Il valore di **R6** viene copiato nel registro **RB**.
3. **Esecuzione dell'operazione:** Il valore di **RA** viene passato all'ingresso **InA** dell'ALU. Il valore immediato **X** passa dalla linea di ingresso "1" del **MuxB** e poi viene inviato all'ingresso **InB** dell'ALU. L'ALU esegue la somma tra **RA** e **X**. Il risultato della somma viene conservato nel registro **RZ**. Il contenuto di **RB** (che è **R6**) viene copiato nel registro **RM**.
4. **Memorizzazione del risultato:** Il valore di **RZ** (l'indirizzo di memoria) viaggia attraverso il bus degli indirizzi. I dati contenuti in **RM** (che erano in **R6**) vengono scritti in memoria all'indirizzo specificato da **RZ**.
5. **Scrittura in memoria:** Nessuna azione viene eseguita in questo stadio.

4.4.3 Salti

Distinguiamo tra salto incondizionato e salto condizionato:

Salto incondizionato:

1. Copio PC nell'indirizzo di memoria, si legge l'istruzione dalla memoria e si copia nei dati del **IR**, si incrementa PC di 4
2. Decodifica istruzione
3. Aumento di PC con spiazzamento. Il valore immediato (spiazzamento) viene decodificato dall'istruzione e viene aggiunto al **PC** per ottenere il nuovo indirizzo di salto.
4. Nessuna azione
5. Nessuna azione

Salto condizionato:

1. Copio PC nell'indirizzo di memoria, si legge l'istruzione dalla memoria e si copia nei dati del **IR**, infine incremento PC di 4
2. Decodifica istruzione, copio R5 in RA, copio R6 in RB
3. Confronto tra RA ed RB. Se la condizione di salto è soddisfatta, il **PC** viene aumentato con il valore immediato (spiazzamento) decodificato dall'istruzione, altrimenti il **PC** rimane invariato.
4. Nessuna azione
5. Nessuna azione

4.4.4 In attesa della memoria

Il tempo di esecuzione degli stadi varia in base al numero di cicli di clock. Poiché la memoria è più lenta del processore e richiede più di un ciclo di clock per essere letta, viene utilizzata una **memoria cache** interna al processore. La cache ha un tempo di accesso che coincide con il ciclo di clock per la lettura.

Se il dato richiesto non si trova in cache, il circuito di controllo estende il passo di esecuzione fino al completamento della lettura dalla memoria principale. Quando la lettura dalla memoria è completa, il segnale **MFC** (Memory Function Completed) viene impostato a 1, indicando che il dato è stato fornito e il processo può proseguire.

4.5 Segnali di controllo

I **circuiti di controllo** emettono segnali che sono essenziali per gestire il corretto funzionamento del processore. Questi segnali abilitano i registri e i multiplexer, selezionando gli input da inviare ai vari componenti.

I registri **PC** e **IR** non devono essere alterati ad ogni ciclo di clock, quindi vengono abilitati solo nei passi in cui è necessaria una scrittura, grazie ai segnali di controllo.

Nel **data path**, i segnali di controllo forniscono l'input di selezione per i multiplexer **MuxB**, **MuxY** e **MuxC**. Inoltre, nella fase di prelievo dell'istruzione, i segnali di controllo selezionano l'ingresso per **MuxMA**, che varia tra lo stadio 1 (quando si usa il **PC** per prelevare l'istruzione) e lo stadio 4 (quando si usa **RZ** per le operazioni di **Load** e **Store**).

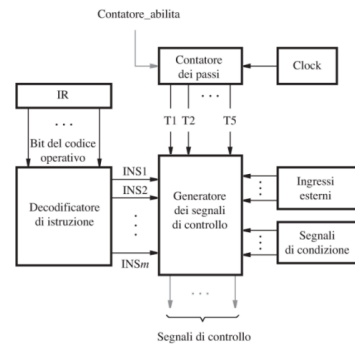
Nel **generatore di indirizzi**, i segnali di controllo gestiscono la selezione per **MuxINC** e **MuxPC**. Inoltre, è necessario inviare un segnale di scrittura al **banco dei registri**, selezionare l'operazione dell'**ALU**, determinare il bit di esito, inviare il segnale di lettura o scrittura alla memoria e gestire il segnale **MFC** (Memory Function Completed).

componenti elettroniche. Generalmente più veloce rispetto al controllo microprogrammato, poiché i segnali di controllo sono generati direttamente dai circuiti logici senza bisogno di accesso alla memoria

4.6.1 Generazione di segnali di controllo

Essenzialmente i circuiti di controllo sono composti da 2 parti:

- Decodificatore di istruzioni



- Generatore di segnali di controllo

Il registro IR è collegato al decodificatore, che prende i bit necessari per prelevare l'op code. In base all'istruzione decodificata manderà un segnale alto nelle linee $INS1, \dots, INS_n$.

Il clock manda il suo segnale al contatore di passi che decodificherà in base al fronte di salita, il passo che si sta eseguendo. Il segnale di clock, insieme a quello MFC sono collegati al segnale di controllo **CONTATORE_ABILITA**, che dirà al contatore di passi di andare al successivo se il precedente è stato completato (MFC alto) e se è stato ricevuto il clock successivo.

Il contatore di passi pone a 1 una uscita da T1 a T5 per indicare quale passo corrente è in corso (tutte le istruzioni si eseguono in 5 passi). Il generatore ha bisogno di sapere il passo in cui siamo, per abilitare i segnali di controllo adatti

4.6.2 Segnali di controllo nel percorso dati

Le impostazioni dei segnali di controllo vengono determinate in base alle azioni che si svolgono in ogni passo di esecuzione di ciascuna istruzione. Di seguito vengono descritti alcuni esempi:

Esempio: Segnale di scrittura sul banco dei registri (RF_scrittura) Il segnale di controllo **RF_scrittura** è attivato (vale 1) nel passo 5 **T5** per tutte le istruzioni che scrivono dati su registri. La sua attivazione dipende dalle operazioni che necessitano di un registro di destinazione. La relazione logica che descrive l'attivazione del segnale è:

- **RF_scrittura = T5 AND (ALU OR Load OR Call)** Ciò significa che **RF_scrittura** è attivo quando:
 1. Il passo 5 è attivo (**T5 = 1**).
 2. L'operazione in corso è un'operazione **Aritmetico-Logica (ALU)**, una **Load** o una **Call**, che necessitano di un registro di destinazione.

Questo segnale viene quindi sintetizzato in porte logiche e, una volta generato, viene inviato dal **generatore di segnali**.

Esempio: Segnale di selezione del bus (B_selettore) Al contrario, il segnale di controllo **B_selettore** non dipende dal passo di esecuzione, ma viene attivato in base alla presenza di un valore **immediato** nell'istruzione.

La relazione logica che descrive l'attivazione del segnale è:

- **B_selettore = Immediato** Questo significa che **B_selettore** è attivo (vale 1) per tutte le istruzioni che contengono un valore immediato nel **registro dell'istruzione (IR)**.

4.6.3 Ritardi della memoria

Il **contatore di passi** viene incrementato alla fine di ogni ciclo di clock, eccetto nei cicli in cui si deve aspettare il completamento di un'operazione di memoria. In tali cicli, si usa il segnale **MFC** (Memory Function Complete) per indicare che l'operazione di memoria è completata.

Per estendere la durata di un passo di esecuzione, è necessario disabilitare il contatore di passi. Ciò avviene tramite il segnale di controllo **CONTATORE_ABILITA**, definito dalla seguente espressione logica:

- **CONTATORE_ABILITA = NOT(WMFC) + MFC** Dove:

- **WMFC** è il segnale che indica che il sistema sta aspettando il completamento dell'operazione di memoria (cioè, **MFC** è ancora basso).
- **MFC** è il segnale che indica che l'operazione di memoria è stata completata (cioè, **MFC** è alto).

Questa logica assicura che il contatore di passi venga abilitato solo nei seguenti casi:

1. Quando non è necessario aspettare (quindi **WMFC** è basso).
2. Quando l'operazione di memoria è completa (cioè **MFC** è alto).

Abilitazione del Program Counter (PC_abilita)

Per l'abilitazione del **Program Counter (PC)**, che gestisce il flusso di esecuzione del programma, si utilizza il seguente segnale:

- $PC_abilita = T1 \cdot MFC + T3 \cdot BR$ Dove:
- **T1** e **T3** sono i passi di esecuzione.
- **MFC** segnala che l'operazione di memoria è completata.
- **BR** è l'insieme di istruzioni che causano un salto (branch).

In sintesi, il **contatore di passi** si abilita quando non si aspetta memoria o l'operazione è finita, mentre il **PC** è abilitato durante il primo passo o quando c'è un salto.

4.7 Processori CISC

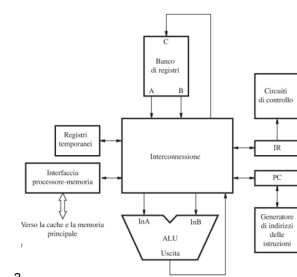
I processori CISC hanno un'organizzazione hardware più complessa, poiché:

- Hanno modi di indirizzamento e operandi vari rispetto a un RISC
- È possibile fare operazioni con operandi in memoria
- Le istruzioni hanno lunghezza variabile, non sono sempre di una parola di memoria

4.7.1 Organizzazione CISC

L'hardware di un processore CISC è composto da queste componenti:

- Banco registri
- ALU
- Registri temporanei (sostituiscono quelli interstadio), mantiene alcuni risultati intermedi
- Blocco interconnessione, che collega tutte le componenti fra loro, trasferisce dati in modo bidirezionale, ed infatti è formato da bus.



4.7.2 Controllo di accesso al bus

Quando più dispositivi in un'architettura CISC sono collegati alla stessa linea di bus, potrebbero tentare di imporre valori simultaneamente, causando conflitti elettrici e corruzione dei dati.

Soluzione: Pilota del Bus a 3 Stati

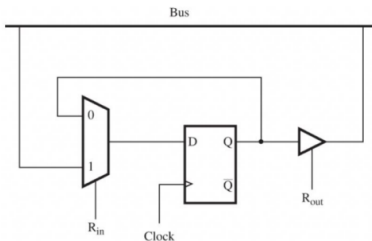
Si utilizza un circuito composto da:

- **Flip-flop di tipo D** (memorizzazione dati)
- **Multiplexer** (controllo ingresso)
- **Porta a 3 stati** (controllo uscita)

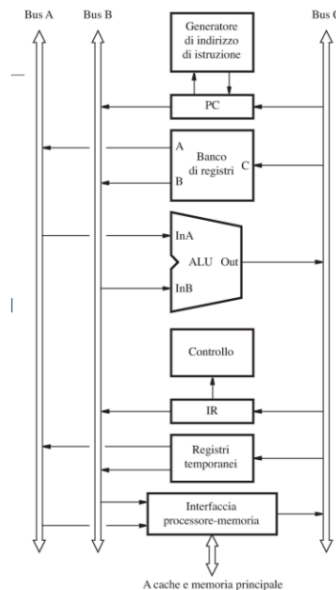
1. Modalità Lettura (Rin = 1)

- Il multiplexer seleziona il dato dal bus

- Al fronte di clock, il flip-flop memorizza il valore
2. **Modalità Mantenimento ($R_{in} = 0$)**
 - Il multiplexer isola il flip-flop dal bus
 - Il flip-flop conserva il suo valore corrente
 3. **Modalità Scrittura ($R_{out} = 1$)**
 - La porta a 3 stati è abilitata
 - Il valore del flip-flop viene posto sul bus
 4. **Modalità Disconnessione ($R_{out} = 0$)**
 - La porta a 3 stati va in alta impedenza (Z)
 - Il dispositivo risulta elettricamente isolato dal bus



4.7.3 Interconnessione a tre bus



Il blocco interconnessione è formato da tre bus: A, B e C

ADD R5, R6 (3 passi)

Nel caso di un'operazione semplice come **ADD R5, R6**, il processore impiega solo **3 passi** per completarla:

1. **Fetch dell'istruzione:**
 - Il Program Counter (PC) invia l'indirizzo di memoria al **Bus B**, che lo passa all'interfaccia processore-memoria.
 - L'interfaccia legge l'istruzione e la invia al **Bus C**, che la carica nel **Registro IR** (Instruction Register).
2. **Decodifica e lettura dei registri:**
 - L'unità di controllo decodifica l'istruzione e attiva i segnali per leggere i valori di **R5 e R6** dal banco registri.
3. **Esecuzione e scrittura del risultato:**
 - I valori di **R5 e R6** vengono inviati rispettivamente al **Bus A e Bus B**, che li portano all'ALU.
 - L'ALU esegue la somma e invia il risultato al **Bus C**, che lo scrive in **R5** alla fine del ciclo di clock.

AND X(R7), R9 (7 passi)

Un'operazione più complessa, come **AND X(R7), R9**, richiede invece **7 passi** perché coinvolge:

- **Accesso multiplo alla memoria** (fetch a 2 parole, lettura e scrittura dati)
- **Calcolo di un indirizzo effettivo** ($X + [R7]$)
- **Operazione logica con dato in memoria**

1. Fetch della prima parola istruzione:

- Il PC invia l'indirizzo al **Bus B**, l'interfaccia legge l'opcode e lo carica nell'**IR**.
- Il PC si incrementa di **+4** per prepararsi a leggere lo **spiazzamento X** (32 bit).

2. Decodifica e lettura registri:

- Vengono generati i segnali di controllo per leggere **R7 e R9**.
- Si avvia la lettura della prima parola di memoria.

3. Fetch dello spiazzamento X:

- Il PC (ora puntato alla seconda parola) legge **X** e lo salva in un registro temporaneo (**TEMP1**).
- Il PC si incrementa nuovamente di **+4** per proseguire con l'esecuzione.

4. Calcolo dell'indirizzo effettivo ($X + [R7]$):

- I valori di **R7** e **TEMP1 (X)** vengono inviati all'ALU tramite i **Bus A e B**.
- L'ALU li somma e salva il risultato in **TEMP2** (indirizzo effettivo).

5. Lettura del dato in memoria (locazione $X+R7$):

- L'indirizzo in **TEMP2** viene inviato al **Bus B** per leggere la memoria.
- Il dato letto viene salvato in **TEMP1**, sovrascrivendo il precedente valore (X).

6. Esecuzione dell'AND logico:

- Il valore di **R9** e il dato in **TEMP1** (letto da memoria) vengono inviati all'ALU.
- L'ALU esegue l'operazione **AND** e salva il risultato in **TEMP1**.

7. Scrittura del risultato in memoria:

- Il risultato (**TEMP1**) e l'indirizzo (**TEMP2**) vengono inviati all'interfaccia memoria.
- Il dato viene scritto nella locazione $X + [R7]$, completando l'istruzione.

4.7.4 Controllo microprogrammato

Il controllo microprogrammato in un processore **CISC** (Complex Instruction Set Computing) è un metodo utilizzato per generare i segnali di controllo necessari all'esecuzione delle istruzioni.

1. **Parola di controllo (control word):** È l'insieme di segnali di controllo necessari per attivare i componenti della CPU durante un passo di esecuzione. Ogni passo di esecuzione di un'istruzione corrisponde a una parola di controllo.
2. **Microistruzione:** Ogni microistruzione rappresenta uno specifico set di segnali di controllo. Indica quali componenti della CPU devono essere attivati e quali operazioni devono essere eseguite durante un passo di esecuzione.
3. **Microroutine:** È una sequenza di microistruzioni che vengono eseguite in una sequenza temporale per completare una determinata operazione.
4. **Microprogramma:** È un programma che consiste di una serie di microroutine, ciascuna delle quali definisce i passi necessari per l'esecuzione delle istruzioni del set di istruzioni (ISA) della CPU.

Nei processori **CISC**, le prime due microistruzioni (prelievo e decodifica) sono comuni per tutte le istruzioni. Le microroutine successive variano in base all'operazione specifica da eseguire. L'interprete delle istruzioni, cioè il microprogramma, gestisce l'intero ciclo di esecuzione delle istruzioni tramite queste sequenze di microistruzioni.

Unità di controllo microprogrammato

La **memoria di controllo** in un processore **CISC** contiene le **microroutine**, sequenze di **microistruzioni** che definiscono i segnali di controllo necessari per eseguire le istruzioni. Il **PC** indirizza le microistruzioni, aggiornato dal **generatore di microindirizzi**. Il **registro delle istruzioni (IR)** decodifica il codice operativo e la modalità di indirizzamento, trovando l'indirizzo della microroutine. Ogni **microistruzione** nella microroutine attiva i segnali di controllo necessari per completare l'operazione.

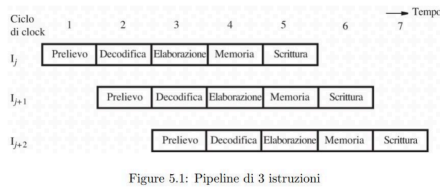
Controllo cablato vs microprogrammato

Il controllo microprogrammato è semplice da realizzare rispetto al cablato, ma è anche più lento rispetto al controllo cablato, poiché ogni microistruzione richiede un ciclo di clock per l'accesso alla memoria di controllo. Il costo dei circuiti logici non è più un fattore significativo; quindi, il controllo cablato è diventata la scelta preferita.

CAPITOLO 5 - INTRODUZIONE AL PIPELINING

"Pipelining" vuol dire eseguire istruzioni diverse in parallelo. Senza pipeline ogni istruzione prenderebbe ognuna 5 cicli di clock. Eseguendo un'istruzione in pipeline, ad ogni ciclo vengono eseguite contemporaneamente diverse istruzioni, le quali

si trovano in diversi stadi di esecuzione.



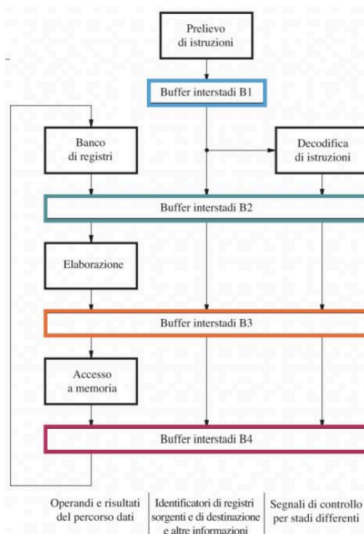
Se avessimo eseguito sequenzialmente le istruzioni, avremmo dovuto aspettare 15 cicli di clock, ma con la pipeline ne aspettiamo solo 7.

5.1 Organizzazione in pipeline

Con l'organizzazione in pipelining, il datapath viene leggermente modificato: al posto dei registri interstadio vi sono i buffer interstadio, che contengono i registri interstadio e una copia del registro IR e PC, e dei segnali di controllo. I buffer alimentano lo stadio successivo trasferendo le informazioni.

- Il Buffer B1 alimenta lo stadio Decodifica
- Il Buffer B2 alimenta lo stadio Elaborazione con i due operandi letti dal banco registri, il valore immediato, PC e i segnali di controllo.
- Il Buffer B3 tiene il risultato del calcolo svolto dall'ALU e nel caso di scrittura in memoria lo scrive.
- Il Buffer B4 alimenta lo stadio scrittura con un valore da scrivere nel banco registri (risultato ALU o dell'accesso in memoria)

5.1.1 Percorso dati nel pipelining



1. Primo ciclo di clock (prelievo della prima istruzione - ADD):

- **B1:** PC = 101, IR = ADD
- Viene prelevata l'istruzione ADD e trasferita nel registro IR.
- Il **PC** punta all'indirizzo della prossima istruzione.

2. Secondo ciclo di clock (prelievo della seconda istruzione - OR, decodifica della prima - ADD):

- **B1:** PC = 102, IR = OR
- **B2:** contiene i dati del primo ciclo (PC = 101, IR = ADD), con **RA** (uscita del banco registri) che contiene il valore del registro R3.
- L'istruzione ADD è ora in fase di decodifica e l'operazione viene preparata.

3. Terzo ciclo di clock (prelievo della terza istruzione - SUB, decodifica della seconda - OR, elaborazione della prima - ADD):

- **B1:** PC = 103, IR = SUB
- **B2:** contiene i dati trasferiti da B1 (PC = 102, IR = OR, **RA** = [R5], **RB** = [R6]).
- **B3:** contiene i dati trasferiti da B2 (PC = 101, IR = ADD, **RZ** = [R3] + 100, risultato dell'ALU).

4. Quarto ciclo di clock (prelievo della quarta istruzione, decodifica della terza - SUB, elaborazione della seconda - OR, accesso in memoria per la prima - ADD):

- **B1**: PC = 104, IR = prossima istruzione.
 - **B2**: PC = 103, IR = SUB, **RA** = [R2].
 - **B3**: contiene i dati trasferiti da B2 (PC = 102, IR = OR, **RZ** = [R5] OR [R6]).
 - La memoria è acceduta per la prima istruzione (ADD), scrivendo il valore in **R2**.
5. **Quinto ciclo di clock (prelievo della quinta istruzione, decodifica della quarta, elaborazione della terza, memoria della seconda, scrittura della prima):**
- **B1**: PC = 105, IR = prossima istruzione.
 - **B2**: contiene il nuovo valore di PC = 104 e la prossima istruzione da decodificare.
 - **B3**: contiene i dati (PC = 103, IR = SUB, **RZ** = [R2] - 30).
 - **B4**: contiene i dati (PC = 102, IR = OR, **RY** = [R5] OR [R6]).
 - La scrittura del risultato dell'operazione ADD avviene in **R2**.

In sintesi, il ciclo di esecuzione prevede che ogni istruzione venga prelevata, decodificata, elaborata e scritta nel registro di destinazione, mentre il ciclo di clock gestisce il trasferimento dei dati tra i registri (B1, B2, B3) e l'esecuzione delle operazioni richieste.

5.2 Problematiche del pipelining

Il caso ideale di sovrapposizione degli stadi di esecuzione di istruzioni diverse non è sempre realizzabile nella pratica. Ad esempio, può capitare che un'istruzione abbia come registro sorgente il registro destinazione di una istruzione precedente, e quindi deve aspettare che i dati vengano scritti, rimanendo nello stato di decodifica. In questo caso avviene un conflitto, o un **hazard**, che è una qualunque situazione che causa un rallentamento di esecuzione. Questa può dipendere appunto da una dipendenza di dato, ritardi in memoria, istruzioni di salto o limiti di risorse.

5.2.1 Dipendenza di dato

Possiamo risolvere il conflitto nei seguenti modi:

- Possiamo generare con i segnali di controllo, dei cicli di attesa (in questo caso 3), attraverso NOP, cioè no operation, per rallentare l'esecuzione fino a quando ne abbiamo bisogno. Il metodo è chiamato BUBBLE (bolle virtuali per visualizzare che in quei cicli di clock non è stato fatto nulla)